

1. Introduction to Python - សេចក្តីផ្តើមអំពី Python

Python គឺជាភាសាសរសេរកម្មវិធីដែលមានភាពបត់បែនខ្ពស់ និងងាយស្រួលសម្រាប់អ្នកចាប់ផ្តើមរៀន។ វាត្រូវបានប្រើប្រាស់យ៉ាងទូលំទូលាយក្នុងការអភិវឌ្ឍគេហទំព័រ ការវិភាគទិន្នន័យ បញ្ហាសិប្បនិម្មិត និងការងារជាច្រើនផ្សេងទៀត។ ផ្នែកនេះនឹងផ្តល់នូវទិដ្ឋភាពទូទៅអំពី Python និងរបៀបចាប់ផ្តើមប្រើប្រាស់វា។

1.1 What is Python? - Python ជាអ្វី?

Python គឺជាភាសាសរសេរកម្មវិធី high-level ដែលត្រូវបានបង្កើតឡើងដោយ Guido van Rossum នៅឆ្នាំ 1991។ វាល្បីល្បាញដោយសារមាន syntax សាមញ្ញ និងអាចអានបានយ៉ាងងាយស្រួល ដែលធ្វើឱ្យវាស្ថិតសម្រាប់អ្នករៀនថ្មី។

លក្ខណៈពិសេស៖

- Syntax សាមញ្ញ និងងាយស្រួលអាន។
- គាំទ្រម្រង់សរសេរកម្មវិធីច្រើនដូចជា object-oriented និង functional។
- មាន libraries និង tools ជាច្រើនសម្រាប់ការអភិវឌ្ឍ។

1.2 Why Learn Python? - ហេតុអ្វីត្រូវរៀន Python?

ការរៀន Python មានសារៈសំខាន់ព្រោះវាជាភាសាដែលងាយស្រួលរៀន និងមានតម្រូវការខ្ពស់នៅក្នុងទីផ្សារការងារ។ វាអាចប្រើប្រាស់បានសម្រាប់ការងារផ្សេងៗដូចជា ការសរសេរកម្មវិធី ការវិភាគទិន្នន័យ និង machine learning។ លើសពីនេះ Python មានសហគមន៍អ្នកប្រើប្រាស់ដ៏ធំដែលផ្តល់ការគាំទ្រ និងធនធានសិក្សាជាច្រើន។

1.3 Installing Python (Windows, macOS, Linux) - ការដំឡើង Python (Windows, macOS, Linux)

ដើម្បីប្រើ Python អ្នកត្រូវដំឡើងវានៅលើកុំព្យូទ័ររបស់អ្នកជាមុនសិន។ ជំហានខាងក្រោមនេះនឹងណែនាំពីរបៀបដំឡើង Python នៅលើប្រព័ន្ធប្រតិបត្តិការ Windows, macOS និង Linux។

ជំហានដំឡើង៖

1. ចូលទៅកាន់គេហទំព័រផ្លូវការ <https://www.python.org/downloads/> ហើយទាញយកកំណែចុងក្រោយបំផុត។
2. បើកឯកសារដែលបានទាញយក ហើយធ្វើតាមការណែនាំតាមប្រព័ន្ធប្រតិបត្តិការរបស់អ្នក។
3. បន្ទាប់ពីដំឡើងរួច ពិនិត្យកំណែ Python ដោយប្រើពាក្យបញ្ជា៖

```
python --version
```

ឧទាហរណ៍លទ្ធផល៖ Python 3.11.4

1.4 Running Python Programs (IDLE, VS Code, PyCharm) - ការដំណើរការកម្មវិធី Python (IDLE, VS Code, PyCharm)

បន្ទាប់ពីដំឡើង Python អ្នកអាចដំណើរការកម្មវិធី Python បានតាមវិធីផ្សេងៗដូចជា ប្រើ IDLE (ឧបករណ៍ដែលភ្ជាប់មកជាមួយ Python), Visual Studio Code (VS Code), ឬ PyCharm។ ខាងក្រោមនេះគឺជាឧទាហរណ៍មួយដោយប្រើ command line។

ឧទាហរណ៍ការដំណើរការ៖

```
# បង្កើតឯកសារឈ្មោះ hello.py
echo "print('Hello, World!')" > hello.py

# ដំណើរការឯកសារ hello.py
python hello.py
```

ការពន្យល់៖ ពាក្យបញ្ជាខាងលើបង្កើតឯកសារមួយឈ្មោះ hello.py ដែលមានកូដ Python សម្រាប់បោះពុម្ព "Hello, World!"។ បន្ទាប់មក យើងប្រើ python hello.py ដើម្បីដំណើរការវា។

លទ្ធផល៖

```
Hello, World!
```

ប្រសិនបើអ្នកប្រើ IDLE, VS Code, ឬ PyCharm អ្នកអាចសរសេរកូដនៅក្នុង editor របស់ពួកគេ ហើយចុច "Run" ដើម្បីដំណើរការវាដោយផ្ទាល់។

2. Python Basics (Syntax & Variables) - មូលដ្ឋានគ្រឹះ Python (Syntax និង Variables)

ផ្នែកនេះនឹងណែនាំអំពីមូលដ្ឋានគ្រឹះនៃ Python ដែលរួមមាន syntax និង variables។ វាជាជំហានដំបូងដ៏សំខាន់សម្រាប់អ្នកចាប់ផ្តើមរៀន Python ដើម្បីយល់ពីរបៀបសរសេរកូដឱ្យត្រឹមត្រូវ។

2.1 Python Syntax & Indentation - Syntax និង Indentation របស់ Python

Python ប្រើ indentation (ការចូលបន្ទាត់) ដើម្បីកំណត់ blocks នៃកូដ ជំនួសឱ្យការប្រើ curly braces ដូចភាសាផ្សេងទៀត។ Syntax របស់ Python ត្រូវបានរចនាឱ្យសាមញ្ញ និងងាយយល់។

ឧទាហរណ៍៖

```
# ឧទាហរណ៍ syntax និង indentation
if 5 > 2:
    print("5 ធំជាង 2")
else:
    print("5 មិនធំជាង 2")
```

ការពន្យល់៖ នៅក្នុងឧទាហរណ៍ខាងលើ កូដក្រោម if និង else ត្រូវបានចូលបន្ទាត់ (indent) ដើម្បីបង្ហាញថាវាជា block មួយ។ បើគ្មាន indentation ត្រឹមត្រូវ Python នឹងបង្ហាញ error។

លទ្ធផល៖

```
5 ធំជាង 2
```

2.2 Variables & Constants - Variables និង Constants

Variables គឺជាទីតាំងផ្ទុកទិន្នន័យដែលអាចផ្លាស់ប្តូរបាន ចំណែក constants គឺជាតម្លៃដែលមិនផ្លាស់ប្តូរ។ នៅក្នុង Python អ្នកបង្កើត variable ដោយកំណត់ឈ្មោះ និងតម្លៃដោយប្រើសញ្ញា = ។

ឧទាហរណ៍៖

```
# Variables
name = "Sok"
age = 25

# Constants (ជាទម្លាប់ សរសេរជាអក្សរធំ)
PI = 3.14

print(name)
print(age)
print(PI)
```

ការពន្យល់៖ នៅទីនេះ name និង age ជា variables ដែលអាចផ្លាស់ប្តូរតម្លៃបាន។ PI ជា constant ដែលជាទម្លាប់សរសេរជាអក្សរធំដើម្បីបង្ហាញថាវាមិនគួរផ្លាស់ប្តូរ។

លទ្ធផល៖

```
Sok  
25  
3.14
```

2.3 Data Types (int, float, str, bool, complex) - ប្រភេទទិន្នន័យ (int, float, str, bool, complex)

Python មាន data types ជាច្រើនដូចជា int (ចំនួនគត់), float (ចំនួនទសភាគ), str (អក្សរ), bool (ត្រូវ/ខុស), និង complex (ចំនួនស្មុគស្មាញ)។

ឧទាហរណ៍៖

```
# Data types  
integer = 10          # int  
floating = 3.14       # float  
string = "Hello"     # str  
boolean = True        # bool  
complex_num = 2 + 3j  # complex  
  
print(type(integer))  
print(type(floating))  
print(type(string))  
print(type(boolean))  
print(type(complex_num))
```

ការពន្យល់៖ យើងប្រើ type() ដើម្បីពិនិត្យប្រភេទទិន្នន័យនៃ variable នីមួយៗ។ ឧទាហរណ៍ខាងលើបង្ហាញ data types ផ្សេងៗ។

លទ្ធផល៖

```
<class 'int'>  
<class 'float'>  
<class 'str'>  
<class 'bool'>  
<class 'complex'>
```

2.4 Type Casting (int(), float(), str()) - ការប្តូរប្រភេទទិន្នន័យ (int(), float(), str())

Type casting គឺជាការប្តូរប្រភេទទិន្នន័យពីមួយទៅមួយផ្សេងទៀតដោយប្រើ int(), float(), ឬ str()។

ឧទាហរណ៍៖

```
# Type casting
num_str = "123"
num_int = int(num_str)    # ប្តូរពី str ទៅ int
num_float = float(num_str) # ប្តូរពី str ទៅ float
text = str(456)           # ប្តូរពី int ទៅ str

print(num_int)
print(num_float)
print(text)
```

ការពន្យល់៖ នៅទីនេះ "123" ត្រូវបានប្តូរទៅជា int និង float ហើយ 456 ត្រូវបានប្តូរទៅជា str។

លទ្ធផល៖

```
123
123.0
456
```

2.5 Taking User Input (input()) - ការទទួលយកទិន្នន័យពីអ្នកប្រើ (input())

នៅក្នុង Python អ្នកអាចប្រើ input() ដើម្បីទទួលយកទិន្នន័យពីអ្នកប្រើប្រាស់ ហើយរក្សាទុកវាក្នុង variable។

ឧទាហរណ៍៖

```
# Taking user input
name = input("បញ្ចូលឈ្មោះរបស់អ្នក: ")
age = int(input("បញ្ចូលអាយុរបស់អ្នក: "))

print(f"សួស្តី {name}! អ្នកមានអាយុ {age} ឆ្នាំ។")
```

ការពន្យល់៖ input() ស្នើសុំអ្នកប្រើបញ្ចូលទិន្នន័យ។ យើងប្រើ int() ដើម្បីប្តូរអាយុពី str ទៅ int។

លទ្ធផល (ឧទាហរណ៍បញ្ចូល "Sok" និង "25")៖

បញ្ចូលឈ្មោះរបស់អ្នក: Sok
បញ្ចូលអាយុរបស់អ្នក: 25
ស្អី Sok! អ្នកមានអាយុ 25 ឆ្នាំ។

3. Operators in Python - ប្រមាណវិធីនៅក្នុង Python

Operators គឺជាទីមិត្តសញ្ញាដែលប្រើសម្រាប់ធ្វើប្រតិបត្តិការលើ variables និងតម្លៃនានា។ នៅក្នុង Python មាន operators ជាច្រើនប្រភេទដែលជួយសម្រួលដល់ការសរសេរកម្មវិធី ដូចជា ការគណនា ការប្រៀបធៀប និងការតភ្ជាប់តួ។

3.1 Arithmetic Operators (+, -, *, /, //, %, **) - ប្រមាណវិធីគណិត (+, -, *, /, //, %, **)

Arithmetic operators ត្រូវបានប្រើសម្រាប់ការគណនាគណិតសាស្ត្រដូចជា បូក ដក គុណ និងចែក។

ឧទាហរណ៍៖

```
a = 10
b = 3

print(a + b) # បូក
print(a - b) # ដក
print(a * b) # គុណ
print(a / b) # ចែក (លទ្ធផលជា float)
print(a // b) # ចែកយកតែផ្នែកគត់
print(a % b) # យកសំណល់
print(a ** b) # លើកកម្ពស់ (គុណ b ដង)
```

ការពន្យល់៖ + បូក, - ដក, * គុណ, / ចែក (លទ្ធផលជា float), // ចែកយកតែផ្នែកគត់, % យកសំណល់, ** លើកកម្ពស់។

លទ្ធផល៖

```
13
7
```

```
30
3.3333333333333335
3
1
1000
```

3.2 Comparison Operators (==, !=, >, <, >=, <=) - ប្រមាណវិធីប្រៀបធៀប (==, !=, >, <, >=, <=)

Comparison operators ត្រូវបានប្រើដើម្បីប្រៀបធៀបតម្លៃពីរ ហើយផ្តល់លទ្ធផលជា True ឬ False។ វាមានប្រយោជន៍សម្រាប់ការសម្រេចចិត្តនៅក្នុងកម្មវិធី។

ឧទាហរណ៍៖

```
x = 5
y = 10

print(x == y) # ស្មើគ្នា
print(x != y) # មិនស្មើគ្នា
print(x > y)  # ធំជាង
print(x < y)  # តូចជាង
print(x >= y) # ធំជាងឬស្មើ
print(x <= y) # តូចជាងឬស្មើ
```

ការពន្យល់៖ == ពិនិត្យថាតើតម្លៃស្មើគ្នា, != ពិនិត្យថាតើមិនស្មើគ្នា, ...

លទ្ធផល៖

```
False
True
False
True
False
True
```

3.3 Logical Operators (and, or, not) - ប្រមាណវិធីតក្កៈ (and, or, not)

Logical operators ត្រូវបានប្រើដើម្បីផ្សំលក្ខខណ្ឌច្រើន ហើយផ្តល់លទ្ធផលជា True ឬ False។

ឧទាហរណ៍៖

```
a = True
b = False

print(a and b) # ទាំងពីរត្រូវតែ True
print(a or b)  # យ៉ាងហោចណាស់មួយ True
print(not a)   # ប្តូរពី True ទៅ False ឬផ្ទុយមកវិញ
```

ការពន្យល់៖ and តម្រូវឱ្យលក្ខខណ្ឌទាំងពីរពិត, or តម្រូវឱ្យយ៉ាងហោចណាស់មួយពិត, not ប្តូរតម្លៃតក្កៈ។

លទ្ធផល៖

```
False
True
False
```

3.4 Bitwise Operators (&, |, ^, ~, <<, >>) - ប្រមាណវិធីប៊ីត (&, |, ^, ~, <<, >>)

Bitwise operators ធ្វើការលើ bits នៃចំនួនគត់ (binary) ដើម្បីអនុវត្តប្រតិបត្តិការតក្កៈនៅកម្រិតទាប។

ឧទាហរណ៍៖

```
x = 5 # binary: 0101
y = 3 # binary: 0011

print(x & y) # AND (0101 & 0011 = 0001)
print(x | y) # OR (0101 | 0011 = 0111)
print(x ^ y) # XOR (0101 ^ 0011 = 0110)
print(~x)    # NOT (~0101 = -0110)
print(x << 1) # Left shift (0101 << 1 = 1010)
print(x >> 1) # Right shift (0101 >> 1 = 0010)
```

ការពន្យល់៖ & ធ្វើ AND, | ធ្វើ OR, ^ ធ្វើ XOR, ~ ប្តូរប៊ីត, << ផ្លាស់ទៅឆ្វេង, >> ផ្លាស់ទៅស្តាំ។

លទ្ធផល៖

```
1
7
6
```



```
-6
10
2
```

3.5 Assignment Operators (=, +=, -=, etc.) - ប្រមាណវិធីកំណត់តម្លៃ (=, +=, -=, ជាដើម)

Assignment operators ត្រូវបានប្រើដើម្បីកំណត់តម្លៃទៅ variables ឬធ្វើប្រតិបត្តិការរួមជាមួយការកំណត់តម្លៃ។

ឧទាហរណ៍៖

```
x = 10
print(x)

x += 5 # x = x + 5
print(x)

x -= 3 # x = x - 3
print(x)

x *= 2 # x = x * 2
print(x)

x /= 4 # x = x / 4
print(x)
```

ការពន្យល់៖ = កំណត់តម្លៃ, += បូកហើយកំណត់, -= ដកហើយកំណត់, *= គុណហើយកំណត់, /= ចែកហើយកំណត់។

លទ្ធផល៖

```
10
15
12
24
6.0
```

4. Control Flow (Conditions & Loops) - លំហូរគ្រប់គ្រង (លក្ខខណ្ឌ និង រង្វិលជុំ)

Control flow នៅក្នុង Python អនុញ្ញាតឱ្យអ្នកគ្រប់គ្រងលំដាប់នៃការប្រតិបត្តិកូដដោយប្រើលក្ខខណ្ឌ (conditions) និងរង្វិលជុំ (loops)។ ផ្នែកនេះនឹងពន្យល់ពីរបៀបប្រើ if, if else, if elif else, for, និង while ដើម្បីធ្វើឱ្យកម្មវិធីមានភាពបត់បែន។

4.1 if-else Statements - សេចក្តីបញ្ជា if-else

if សេចក្តីបញ្ជាត្រូវបានប្រើដើម្បីសម្រេចចិត្តផ្អែកលើលក្ខខណ្ឌ។ វាអាចប្រើតែ if តែមួយ ឬរួមបញ្ចូលជាមួយ else និង elif ដើម្បីគ្រប់គ្រងលទ្ធផលផ្សេងៗ។ ប្រសិនបើលក្ខខណ្ឌពិត (True) កូដនឹងប្រតិបត្តិផ្នែកមួយ បើមិនដូច្នោះទេ វានឹងប្រតិបត្តិផ្នែកផ្សេង។

4.1.1 Simple if Statement - សេចក្តីបញ្ជា if សាមញ្ញ

if សាមញ្ញត្រូវបានប្រើនៅពេលអ្នកចង់ប្រតិបត្តិកូដតែនៅពេលលក្ខខណ្ឌមួយពិត។

ឧទាហរណ៍៖

```
temperature = 30

if temperature > 25:
    print("អាកាសធាតុក្ដៅណាស់")
```

ការពន្យល់៖ if temperature > 25 ពិនិត្យថាតើសីតុណ្ហភាពធំជាង 25, បើពិត បោះពុម្ព 'អាកាសធាតុក្ដៅណាស់', បើមិនពិត មិនធ្វើអ្វីទេ។

លទ្ធផល៖

```
អាកាសធាតុក្ដៅណាស់
```

4.1.2 if-else Statement - សេចក្តីបញ្ជា if-else

if-else បន្ថែមផ្នែក else ដើម្បីប្រតិបត្តិកូដនៅពេលលក្ខខណ្ឌមិនពិត (False)។

ឧទាហរណ៍៖

```
age = 18

if age >= 18:
```

```
print("អ្នកជាមនុស្សពេញវ័យ")
else:
    print("អ្នកនៅតែជាអនីតិជន")
```

ការពន្យល់៖ if age >= 18 ពិនិត្យថា តើអាយុធំជាងឬស្មើ 18, បើពិត បោះពុម្ព 'អ្នកជាមនុស្សពេញវ័យ', បើមិនពិត បោះពុម្ព 'អ្នកនៅតែជាអនីតិជន'។

លទ្ធផល៖

អ្នកជាមនុស្សពេញវ័យ

4.1.3 if-elif-else Statement - សេចក្តីបញ្ជា if-elif-else

if-elif-else អនុញ្ញាតឱ្យពិនិត្យលក្ខខណ្ឌច្រើន ដោយប្រើ elif (else if) សម្រាប់លក្ខខណ្ឌបន្ថែម និង else សម្រាប់ករណីផ្សេងទៀតទាំងអស់។

ឧទាហរណ៍៖

```
score = 75

if score >= 90:
    print("អ្នកបាននិទ្ទេស A")
elif score >= 80:
    print("អ្នកបាននិទ្ទេស B")
elif score >= 70:
    print("អ្នកបាននិទ្ទេស C")
elif score >= 60:
    print("អ្នកបាននិទ្ទេស D")
else:
    print("អ្នកបាននិទ្ទេស F")
```

ការពន្យល់៖ if score >= 90 ពិនិត្យថា តើពិន្ទុធំជាងឬស្មើ 90, បើពិត បោះពុម្ព 'និទ្ទេស A', elif score >= 80 ពិនិត្យបន្ទាប់បើធំជាងឬស្មើ 80, បើពិត បោះពុម្ព 'និទ្ទេស B', បន្តពិនិត្យលក្ខខណ្ឌបន្ទាប់ៗទៀត, បើគ្មានលក្ខខណ្ឌណាពិត បោះពុម្ព 'និទ្ទេស F'។

លទ្ធផល៖

អ្នកបាននិទ្ទេស C

4.2 Nested if Statements - សេចក្តីបញ្ជា if ខាងក្នុង if

Nested if គឺជា if ដែលស្ថិតនៅក្នុង if មួយទៀត ដើម្បីពិនិត្យលក្ខខណ្ឌបន្ថែម។

ឧទាហរណ៍៖

```
score = 85

if score >= 60:
    print("អ្នកបានជាប់")
    if score >= 80:
        print("អបអរសាទរ! អ្នកបានពិន្ទុល្អណាស់")
else:
    print("អ្នកមិនបានជាប់ទេ")
```

ការពន្យល់៖ if score >= 60 ពិនិត្យថា តើពិន្ទុធំជាងឬស្មើ 60, បើពិត បោះពុម្ព 'អ្នកបានជាប់', ហើយបន្តពិនិត្យ `if score >= 80` ថា តើពិន្ទុធំជាងឬស្មើ 80, បើពិត បោះពុម្ពសារអបអរ, បើមិនមែន រំលងផ្នែកខាងក្នុង។

លទ្ធផល៖

```
អ្នកបានជាប់
អបអរសាទរ! អ្នកបានពិន្ទុល្អណាស់
```

4.3 for Loop & while Loop - រង្វិលជុំ for និង while

for និង while ត្រូវបានប្រើដើម្បីធ្វើការងារដដែលៗ។ for ប្រើសម្រាប់រង្វិលជុំលើលំដាប់ (sequence) ចំណែក while ប្រើសម្រាប់រង្វិលជុំដរាបណាលក្ខខណ្ឌនៅតែពិត។

ឧទាហរណ៍ (for loop)៖

```
for i in range(5):
    print(f"លេខ: {i}")
```

ឧទាហរណ៍ (while loop)៖

```
count = 0
while count < 5:
    print(f"ចំនួន: {count}")
    count += 1
```

ការពន្យល់៖ for i in range(5) រត់ពី 0 ដល់ 4, បោះពុម្ពលេខនីមួយៗ, while count < 5 រត់ដរាបណា count តូចជាង 5, បោះពុម្ព count ហើយបន្ថែម 1។

លទ្ធផល (for loop)៖

```
លេខ: 0
លេខ: 1
លេខ: 2
លេខ: 3
លេខ: 4
```

លទ្ធផល (while loop):

```
ថ្ងៃទី១: 0
ថ្ងៃទី១: 1
ថ្ងៃទី១: 2
ថ្ងៃទី១: 3
ថ្ងៃទី១: 4
```

4.4 Loop Control (break, continue, pass) - ការគ្រប់គ្រងរង្វិលជុំ (break, continue, pass)

break, continue, និង pass ត្រូវបានប្រើដើម្បីគ្រប់គ្រងឥរិយាបថនៃរង្វិលជុំ។ break បញ្ចប់រង្វិលជុំ, continue រំលងទៅរដ្ឋបន្ទាប់, និង pass មិនធ្វើអ្វីទេ (placeholder)។

ឧទាហរណ៍:

```
for i in range(10):
    if i == 5:
        break # បញ្ចប់រង្វិលជុំនៅ i = 5
    if i == 3:
        continue # រំលង i = 3
    if i == 7:
        pass # មិនធ្វើអ្វីទេ
    print(f"លេខ: {i}")
```

ការពន្យល់: break បញ្ចប់រង្វិលជុំនៅ i = 5, continue រំលងការបោះពុម្ពនៅ i = 3, pass មិនធ្វើអ្វីទេនៅ i = 7 ប៉ុន្តែបន្តរង្វិលជុំ។

លទ្ធផល:

```
លេខ: 0
លេខ: 1
```

លេខ: 2

លេខ: 4

5. Functions & Modules - អនុគមន៍ និង ម៉ូឌុល

Functions និង Modules ជួយរៀបចំកូដឱ្យមានរបៀប និងអាចប្រើឡើងវិញបាន។ Functions ជាបណ្តុំកូដដែលអាចហៅប្រើបានច្រើនដង ចំណែក Modules ជាឯកសារដែលផ្ទុក Functions និងកូដផ្សេងទៀត។

5.1 Defining & Calling Functions - ការកំណត់ និង ការហៅអនុគមន៍

Functions ត្រូវបានកំណត់ដោយប្រើ `def` ហើយអាចហៅប្រើបានដោយឈ្មោះរបស់វា។

ឧទាហរណ៍៖

```
def greet():  
    print("សួស្តី! ស្វាគមន៍មកកាន់ Python")  
  
greet()
```

ការពន្យល់៖ `def greet()` កំណត់អនុគមន៍ឈ្មោះ `greet`, នៅពេលហៅ `<code>greet()</code>` វាបោះពុម្ពសារ 'សួស្តី! ស្វាគមន៍មកកាន់ Python'។

លទ្ធផល៖

```
សួស្តី! ស្វាគមន៍មកកាន់ Python
```

5.2 Function Parameters (Positional, Default, Keyword, *args, **kwargs) - ប៉ារ៉ាម៉ែត្រអនុគមន៍

Functions អាចទទួល parameters ដើម្បីធ្វើការជាមួយទិន្នន័យផ្សេងៗ។ មានប្រភេទ parameters ជាច្រើន៖

ឧទាហរណ៍ (Positional Parameters)៖

```
def add(a, b):  
    return a + b
```

```
print(add(3, 5))
```

ឧទាហរណ៍ (Default Parameters)៖

```
def greet(name="ភ្ញៀវ"):  
    print(f"សួស្តី {name}")  
  
greet()  
greet("សុខា")
```

ឧទាហរណ៍ (Keyword Parameters)៖

```
def introduce(name, age):  
    print(f"ឈ្មោះ: {name} អាយុ {age} ឆ្នាំ")  
  
introduce(age=25, name="សុភី")
```

ឧទាហរណ៍ (*args)៖

```
def sum_all(*args):  
    return sum(args)  
  
print(sum_all(1, 2, 3, 4))
```

ឧទាហរណ៍ (**kwargs)៖

```
def info(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
info(name="សុផា", age=30, city="ភ្នំពេញ")
```

ការពន្យល់៖

- Positional ទទួលតម្លៃតាមលំដាប់ (a, b),
- Default ប្រើតម្លៃលំដាប់ដើមបើមិនបញ្ជាក់ (name='ភ្ញៀវ'),
- Keyword អនុញ្ញាតឱ្យបញ្ជាក់តម្លៃដោយឈ្មោះ (age=25),
- *args ទទួលចំនួនតម្លៃមិនកំណត់ជា tuple,
- **kwargs ទទួលចំនួនតម្លៃមិនកំណត់ជា dictionary។

លទ្ធផល៖

```
8
ស្តី ភ្ញៀវ
ស្តី សុខា
ខ្ញុំឈ្មោះ សុភី អាយុ 25 ឆ្នាំ
10
name: សុផា
age: 30
city: ភ្នំពេញ
```

5.3 Lambda Functions - អនុគមន៍ Lambda

lambda ជាអនុគមន៍អនាមិក (anonymous) ដែលកំណត់ក្នុងមួយជួរសម្រាប់ការងារសាមញ្ញ។

ឧទាហរណ៍៖

```
square = lambda x: x ** 2

print(square(5))
```

ការពន្យល់៖ `lambda x: x ** 2` បង្កើតអនុគមន៍ដែលលើកកម្ពស់ `x` ទៅជាការ៉េ, ហៅ `<code>square(5)</code>` ដើម្បីគណនា 5^2 ។

លទ្ធផល៖

```
25
```

5.4 Importing Modules (Built-in & Custom) - ការនាំចូលម៉ូឌុល

Modules អនុញ្ញាតឱ្យប្រើ functions និងកូដពីឯកសារផ្សេងទៀត។ មាន built-in modules (ភ្ជាប់មកជាមួយ Python) និង custom modules (បង្កើតដោយអ្នក)។

ឧទាហរណ៍ (Built-in Module)៖

```
import math

print(math.sqrt(16))
```

ឧទាហរណ៍ (Custom Module)៖ សន្មតថាមានឯកសារ `<code>my_module.py</code>` ដែលមានកូដ៖


```
# my_module.py
def say_hello():
    print("ជម្រាបសួរ!")
```

ហៅប្រើវា៖

```
import my_module

my_module.say_hello()
```

ការពន្យល់៖ import math នាំចូលម៉ូឌុល math ដែលភ្ជាប់មកជាមួយ, math.sqrt(16) គណនាឫសការ៉េនៃ 16, import my_module នាំចូលម៉ូឌុលផ្ទាល់ខ្លួន, my_module.say_hello() ហៅអនុគមន៍ពីម៉ូឌុល។

លទ្ធផល៖

```
4.0
ជម្រាបសួរ!
```

6. Data Structures - រចនាសម្ព័ន្ធទិន្នន័យ

Data Structures នៅក្នុង Python ជាវិធីសាស្ត្រសម្រាប់រៀបចំ និង រក្សាទុកទិន្នន័យ។ មានប្រភេទជាច្រើនដូចជា Lists, Tuples, Sets, និង Dictionaries ដែលប្រើសម្រាប់គោលបំណងផ្សេងៗគ្នា។

6.1 Lists (Dynamic Arrays) - បញ្ជី (អាចប្រែប្រួល)

Lists ជាបញ្ជីដែលអាចផ្លាស់ប្តូរបាន (mutable) និងអាចផ្ទុកទិន្នន័យច្រើនប្រភេទ។

ឧទាហរណ៍៖

```
fruits = ["ផ្លែប៉ោម", "ផ្លែចេក", "ផ្លែឌីឡីក"]
fruits.append("ផ្លែស្រ្តប៊ីរី")
print(fruits)
fruits[1] = "ផ្លែត្រសក់"
print(fruits)
```

ការពន្យល់៖ fruits = ["ផ្លែប៉ោម", ...] បង្កើតបញ្ជី, append() បន្ថែម 'ផ្លែស្ពឺបឺរី' ទៅចុងបញ្ជី, fruits[1] = "ផ្លែត្រសក់" ផ្លាស់ប្តូរធាតុទី 1 (index 1)។

លទ្ធផល៖

```
['ផ្លែប៉ោម', 'ផ្លែចេក', 'ផ្លែឌីឡីក', 'ផ្លែស្ពឺបឺរី']  
['ផ្លែប៉ោម', 'ផ្លែត្រសក់', 'ផ្លែឌីឡីក', 'ផ្លែស្ពឺបឺរី']
```

6.2 Tuples (Immutable Lists) - ទុប្បល (បញ្ជីមិនអាចផ្លាស់ប្តូរ)

Tuples ស្រដៀងនឹង Lists ប៉ុន្តែមិនអាចផ្លាស់ប្តូរបាន (immutable) បន្ទាប់ពីកំណត់។

ឧទាហរណ៍៖

```
colors = ("ក្រហម", "ខៀវ", "លឿង")  
print(colors[0])  
# colors[0] = "បៃតង" # នឹងបណ្តាលឱ្យមានកំហុស
```

ការពន្យល់៖ colors = ("ក្រហម", ...) បង្កើតទុប្បល, colors[0] ចូលប្រើធាតុទី 0, មិនអាចផ្លាស់ប្តូរធាតុបាន ព្រោះវា immutable។

លទ្ធផល៖

```
ក្រហម
```

6.3 Sets (Unique Collections) - សំណុំ (ការប្រមូលផ្តុំតែមួយគត់)

Sets ជាការប្រមូលផ្តុំធាតុតែមួយគត់ (unique) ដែលមិនមានលំដាប់ និងអាចផ្លាស់ប្តូរបាន (mutable)។

ឧទាហរណ៍៖

```
numbers = {1, 2, 3, 3, 4}  
numbers.add(5)  
print(numbers)
```

ការពន្យល់៖ 4 បង្កើតសំណុំ ដោយ 3 ស្មុនត្រូវបានលុបចោល, add(5) បន្ថែម 5 ទៅសំណុំ។

លទ្ធផល៖

```
{1, 2, 3, 4, 5}
```

6.4 Dictionaries (Key-Value Pairs) - រចនាសម្ព័ន្ធ (គូសោ-តម្លៃ)

Dictionaries ជាការប្រមូលផ្តុំគូសោ-តម្លៃ (key-value pairs) ដែលអាចផ្លាស់ប្តូរបាន (mutable)។

ឧទាហរណ៍៖

```
student = {"ឈ្មោះ": "សុខា", "អាយុ": 20, "ជំនាញ": "សរសេរកម្មវិធី"}
student["អាយុ"] = 21
student["សាលា"] = "សាកលវិទ្យាល័យ"
print(student)
```

ការពន្យល់៖ ឈ្មោះ: សុខា, ... បង្កើតរចនាសម្ព័ន្ធ, student["អាយុ"] = 21 កែតម្លៃនៃសោ 'អាយុ', student["សាលា"] = "សាកលវិទ្យាល័យ" បន្ថែមគូសោ-តម្លៃថ្មី។

លទ្ធផល៖

```
{'ឈ្មោះ': 'សុខា', 'អាយុ': 21, 'ជំនាញ': 'សរសេរកម្មវិធី', 'សាលា': 'សាកលវិទ្យាល័យ'}
```

7. String Manipulation - ការរៀបចំអក្សរ

String Manipulation នៅក្នុង Python ជាវិធីសាស្ត្រសម្រាប់ធ្វើការជាមួយអក្សរ (strings) ដូចជា ការធ្វើទ្រង់ទ្រាយ ការផ្លាស់ប្តូរ និងការវិភាគ។ ផ្នែកនេះនឹងបង្ហាញពីវិធីផ្សេងៗដើម្បីគ្រប់គ្រងអក្សរ។

7.1 String Formatting (f-strings, .format()) - ការធ្វើទ្រង់ទ្រាយអក្សរ

String formatting អនុញ្ញាតឱ្យបញ្ចូលតម្លៃអថេរ (variables) នៅក្នុងអក្សរ។ វិធីទូទៅគឺ f-strings និង .format()។

ឧទាហរណ៍ (f-strings)៖

```
name = "សុខា"
age = 25
print(f"ឈ្មោះ: {name} អាយុ: {age} ឆ្នាំ")
```

ឧទាហរណ៍ (.format())៖

```
name = "សុភី"
age = 30
print("ឈ្មោះ: {} អាយុ {} ឆ្នាំ".format(name, age))
```

ការពន្យល់៖ f"... " ប្រើ f-strings ដើម្បីបញ្ចូលតម្លៃដោយផ្ទាល់នៅក្នុងអក្សរ, .format() បញ្ចូលតម្លៃតាមលំដាប់ទៅក្នុង {}។

លទ្ធផល៖

```
ឈ្មោះ: សុខា អាយុ 25 ឆ្នាំ
ឈ្មោះ: សុភី អាយុ 30 ឆ្នាំ
```

7.2 String Methods (upper(), lower(), replace(), split()) - វិធីសាស្ត្រអក្សរ

String methods ជាមុខងារដែលភ្ជាប់មកជាមួយសម្រាប់ផ្លាស់ប្តូរ ឬវិភាគអក្សរ។

ឧទាហរណ៍៖

```
text = "សួស្តី Python"
print(text.upper())
print(text.lower())
print(text.replace("Python", "អ៊ុកសិក្សា"))
print(text.split(" "))
```

ការពន្យល់៖ upper() ប្តូរទៅអក្សរធំទាំងអស់, lower() ប្តូរទៅអក្សរតូចទាំងអស់, replace() ជំនួសអក្សរមួយដោយអក្សរមួយទៀត, split() បំបែកអក្សរទៅជាបញ្ជីដោយប្រើគម្លាត។

លទ្ធផល៖

```
សួស្តី PYTHON
សួស្តី python
សួស្តី អ៊ុកសិក្សា
['សួស្តី', 'Python']
```

7.3 Regular Expressions (re module) - កន្សោមធម្មតា (ម៉ូឌុល re)

re module អនុញ្ញាតឱ្យប្រើ regular expressions (regex) ដើម្បីស្វែងរក ឬផ្ដាច់ផ្ដាច់លំនាំនៅក្នុងអក្សរ។

ឧទាហរណ៍៖

```
import re

text = "ទូរស័ព្ទ: 012-345-6789, អ៊ីមែល: example@email.com"
phone = re.search(r"\d{3}-\d{3}-\d{4}", text)
email = re.search(r"[a-z]+@[a-z]+\.[a-z]+", text)

print(phone.group())
print(email.group())
```

ការពន្យល់៖ import re ទាំចូលម៉ូឌុល re, re.search() ត្រូវបានកំណត់ដំបូងដែលផ្តល់ឱ្យ, r"\d{3}-\d{3}-\d{4}" លំនាំសម្រាប់លេខទូរស័ព្ទ (xxx-xxx-xxxx), r"[a-z]+@[a-z]+\.[a-z]+" លំនាំសម្រាប់អ៊ីមែល, group() ទាញតម្លៃដែលផ្តល់ឱ្យ។

លទ្ធផល៖

```
012-345-6789
example@email.com
```

8. File Handling - ការគ្រប់គ្រងឯកសារ

File Handling នៅក្នុង Python អនុញ្ញាតឱ្យអ្នក សរសេរ និងគ្រប់គ្រងឯកសារនៅលើប្រព័ន្ធ។ ផ្នែកនេះនឹងបង្ហាញពីវិធីធ្វើការជាមួយឯកសារអត្ថបទ CSV និង JSON។

8.1 Reading & Writing Files (open(), .read(), .write()) - ការអាន និង សរសេរឯកសារ

ការអាន និងសរសេរឯកសារប្រើ open() ដើម្បីចូលប្រើឯកសារ .read() ដើម្បីអាន និង .write() ដើម្បីសរសេរ។

ឧទាហរណ៍ (សរសេរ)៖

```
# សរសេរទៅឯកសារ
with open("example.txt", "w", encoding="utf-8") as file:
    file.write("ស្តីពី ពិភពលោក!")
    "
```

ឧទាហរណ៍ (អាន)៖

```
# អានពីឯកសារ
with open("example.txt", "r", encoding="utf-8") as file:
    content = file.read()
print(content)
```

ការពន្យល់៖ `open("example.txt", "w")` បើកឯកសារសម្រាប់សរសេរ (`w = write`), `file.write()` សរសេរអត្ថបទទៅឯកសារ, `open("example.txt", "r")` បើកឯកសារសម្រាប់អាន (`r = read`), `file.read()` អានខ្លឹមសារទាំងមូល, ប្រើ `<code>with</code>` ដើម្បីបិទឯកសារដោយស្វ័យប្រវត្តិ។

លទ្ធផល៖

ស្តីពី ពិភពលោក!

8.2 Handling CSV (csv module) - ការគ្រប់គ្រង CSV

csv module ជួយអាន និងសរសេរទិន្នន័យទៅឯកសារ CSV (Comma-Separated Values)។

ឧទាហរណ៍ (សរសេរ CSV)៖

```
import csv

# សរសេរទៅឯកសារ CSV
with open("students.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow(["ឈ្មោះ", "អាយុ", "ជំនាញ"])
    writer.writerow(["សុខា", 25, "សរសេរកម្មវិធី"])
```

ឧទាហរណ៍ (អាន CSV)៖

```
import csv

# អានពីឯកសារ CSV
with open("students.csv", "r", encoding="utf-8") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

ការពន្យល់៖ `import csv` នាំចូលម៉ូឌុល `csv`, `csv.writer()` បង្កើតអ្នកសរសេរ CSV, `writer.writerow()` សរសេរជួរមួយទៅឯកសារ, `csv.reader()` អានជួរនីមួយៗជាបញ្ជី, `<code>newline=""` ការពារបន្ថែមជួរទទេនៅ Windows។

លទ្ធផល៖

```
['ឈ្មោះ', 'អាយុ', 'ជំនាញ']  
['សុខា', '25', 'សរសេរកម្មវិធី']
```

8.3 Handling JSON (json module) - ការគ្រប់គ្រង JSON

`json module` ជួយអាន និងសរសេរទិន្នន័យជាទម្រង់ JSON (JavaScript Object Notation)។

ឧទាហរណ៍ (សរសេរ JSON)៖

```
import json  
  
data = {"ឈ្មោះ": "សុភិ", "អាយុ": 30, "ទីកន្លែង": "ភ្នំពេញ"}  
with open("data.json", "w", encoding="utf-8") as file:  
    json.dump(data, file, ensure_ascii=False, indent=4)
```

ឧទាហរណ៍ (អាន JSON)៖

```
import json  
  
with open("data.json", "r", encoding="utf-8") as file:  
    loaded_data = json.load(file)  
print(loaded_data)
```

ការពន្យល់៖ `import json` នាំចូលម៉ូឌុល `json`, `json.dump()` សរសេរទិន្នន័យទៅឯកសារ JSON, `ensure_ascii=False` អនុញ្ញាតឱ្យប្រើអក្សរមិនមែន ASCII (ដូចជាខ្មែរ), `indent=4` ធ្វើឱ្យទិន្នន័យមានទម្រង់ស្អាត, `json.load()` អានទិន្នន័យ JSON ទៅជា dictionary។

លទ្ធផល៖

```
{'ឈ្មោះ': 'សុភិ', 'អាយុ': 30, 'ទីកន្លែង': 'ភ្នំពេញ'}
```

9. Object-Oriented Programming (OOP) - ការសរសេរកម្មវិធីតម្រង់ទិសវត្ថុ

Object-Oriented Programming (OOP) ជាវិធីសាស្ត្រសរសេរកម្មវិធីដែលប្រើ `classes` និង `objects` ដើម្បីរៀបចំកូដឱ្យមានរបៀប និងអាចប្រើឡើងវិញបាន។ ផ្នែកនេះនឹងពន្យល់ពីគោលគំនិតសំខាន់ៗនៃ OOP នៅក្នុង Python។

9.1 Classes & Objects - ថ្នាក់ និង វត្ថុ

Classes ជាប្លង់ (blueprint) សម្រាប់បង្កើត `objects` ដែលជាការកើតឡើងជាក់ស្តែងនៃថ្នាក់។

ឧទាហរណ៍៖

```
class Dog:
    def bark(self):
        print("វ៉ូហ្វ! វ៉ូហ្វ!")

my_dog = Dog()
my_dog.bark()
```

ការពន្យល់៖ `class Dog` បង្កើតថ្នាក់ឈ្មោះ `Dog`, `def bark(self)` កំណត់មុខងារសម្រាប់វត្ថុ, `my_dog = Dog()` បង្កើតវត្ថុ `my_dog` ពីថ្នាក់ `Dog`, `my_dog.bark()` ហៅមុខងាររបស់វត្ថុ។

លទ្ធផល៖

```
វ៉ូហ្វ! វ៉ូហ្វ!
```

9.2 Constructors (`__init__`) - អ្នកសាងសង់

`__init__` ជាមុខងារពិសេស (constructor) ដែលរត់ដោយស្វ័យប្រវត្តិនៅពេលបង្កើតវត្ថុ ដើម្បីកំណត់លក្ខណៈដំបូង។

ឧទាហរណ៍៖

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```



```
def introduce(self):
    print(f"ខ្ញុំឈ្មោះ {self.name} អាយុ {self.age} ឆ្នាំ")

person = Person("សុខា", 25)
person.introduce()
```

ការពន្យល់៖ `__init__(self, name, age)` កំណត់លក្ខណៈ `name` និង `age` ទៅពេលបង្កើតវត្ថុ, `self.name` រក្សាទុកតម្លៃ `name` ទៅវត្ថុ, `person = Person("សុខា", 25)` បង្កើតវត្ថុដោយផ្តល់តម្លៃ, `introduce()` ប្រើលក្ខណៈរបស់វត្ថុ។

លទ្ធផល៖

```
ខ្ញុំឈ្មោះ សុខា អាយុ 25 ឆ្នាំ
```

9.3 Inheritance - ការទទួលមរតក

Inheritance អនុញ្ញាតឱ្យថ្នាក់មួយទទួលយកលក្ខណៈ និងមុខងារពីថ្នាក់មួយទៀត។ មានប្រភេទផ្សេងៗនៃការទទួលមរតក។

9.3.1 Single Inheritance - ការទទួលមរតកតែមួយ

ថ្នាក់កូនទទួលមរតកពីថ្នាក់មេតែមួយ។

ឧទាហរណ៍៖

```
class Animal:
    def __init__(self, name):
        self.name = name

class Cat(Animal):
    def meow(self):
        print(f"{self.name} និយាយ: ម៉ៅ! ម៉ៅ!")

cat = Cat("មីមី")
cat.meow()
```

ការពន្យល់៖ `class Cat(Animal)` ទទួលមរតកពី `Animal`, `self.name` មកពីថ្នាក់មេ, `meow()` ជាមុខងារថ្មីរបស់ `Cat`។

លទ្ធផល៖

```
មីមី និយាយ: ម៉ៅ! ម៉ៅ!
```

9.3.2 Multiple Inheritance - ការទទួលមរតកច្រើន

ថ្នាក់កូនទទួលមរតកពីថ្នាក់មេច្រើនជាងមួយ។

ឧទាហរណ៍៖

```
class Flyer:
    def fly(self):
        print("ហោះ!")

class Swimmer:
    def swim(self):
        print("ហែល!")

class Duck(Flyer, Swimmer):
    pass

duck = Duck()
duck.fly()
duck.swim()
```

ការពន្យល់៖ class Duck(Flyer, Swimmer) ទទួលមរតកពី Flyer និង Swimmer, fly() និង swim() មកពីថ្នាក់មេទាំងពីរ។

លទ្ធផល៖

```
ហោះ!
ហែល!
```

9.3.3 Multilevel Inheritance - ការទទួលមរតកច្រើនកម្រិត

ថ្នាក់មួយទទួលមរតកពីថ្នាក់មេ ហើយថ្នាក់មួយទៀតទទួលមរតកពីថ្នាក់នោះបន្ត។

ឧទាហរណ៍៖

```
class Animal:
    def eat(self):
        print("ញ៉ាំ!")

class Dog(Animal):
    def bark(self):
        print("ឡែ!")
```

```
class Puppy(Dog):
    def play(self):
        print("លេង!")

puppy = Puppy()
puppy.eat()
puppy.bark()
puppy.play()
```

ការពន្យល់៖ Puppy ទទួលមរតកពី Dog, Dog ទទួលមរតកពី Animal, ដូច្នេះ Puppy អាចប្រើ `eat()`, `bark()`, និង `play()`។

លទ្ធផល៖

```
ញ៉ាំ!
វ៉ូហ្វ!
លេង!
```

9.3.4 Hierarchical Inheritance - ការទទួលមរតកជាហានុក្រម

ថ្នាក់មេមួយត្រូវបានទទួលមរតកដោយថ្នាក់កូនច្រើន។

ឧទាហរណ៍៖

```
class Animal:
    def eat(self):
        print("ញ៉ាំ!")

class Cat(Animal):
    def meow(self):
        print("ម៉ៅ!")

class Dog(Animal):
    def bark(self):
        print("វ៉ូហ្វ!")

cat = Cat()
dog = Dog()
cat.eat()
cat.meow()
```

```
dog.eat()
dog.bark()
```

ការពន្យល់៖ Cat និង `Dog` ទទួលមរតកពី Animal, `eat()` មាននៅក្នុងទាំង Cat និង Dog, ប៉ុន្តែ `meow()` និង `bark()` ជាលក្ខណៈផ្ទាល់ខ្លួន។

លទ្ធផល៖

```
ញ៉ាំ!
ម៉ៅ!
ញ៉ាំ!
រ៉ូហ្វ!
```

9.3.5 Hybrid Inheritance - ការទទួលមរតកបែបលាយ

ការរួមបញ្ចូលគ្នានៃប្រភេទទទួលមរតកផ្សេងៗ (ឧ. multilevel និង multiple)។

ឧទាហរណ៍៖

```
class Animal:
    def eat(self):
        print("ញ៉ាំ!")

class Flyer:
    def fly(self):
        print("ហោះ!")

class Bird(Animal):
    def chirp(self):
        print("ឆីប! ឆីប!")

class Parrot(Bird, Flyer):
    def talk(self):
        print("ខ្ញុំអាចនិយាយបាន!")

parrot = Parrot()
parrot.eat()
parrot.chirp()
parrot.fly()
parrot.talk()
```

ការពន្យល់៖ Bird ទទួលមរតកពី Animal (single), Parrot ទទួលមរតកពី Bird (multilevel) និង Flyer (multiple), ដូច្នេះ Parrot អាច `<code>eat()</code>`, `<code>chirp()</code>`, `<code>fly()</code>`, និង `<code>talk()</code>`។

លទ្ធផល៖

```
ញ៉ាំ!
ជិប! ជិប!
ហោះ!
ខ្ញុំអាចនិយាយបាន!
```

9.4 Polymorphism - ពហុរូបភាព

Polymorphism អនុញ្ញាតឱ្យវត្ថុផ្សេងគ្នាឆ្លើយតបទៅនឹងមុខងារដូចគ្នាតាមវិធីផ្សេងគ្នា។

ឧទាហរណ៍៖

```
class Cat:
    def sound(self):
        print("ម៉ៅ!")

class Dog:
    def sound(self):
        print("វ៉ូហ្វ!")

def make_sound(animal):
    animal.sound()

cat = Cat()
dog = Dog()
make_sound(cat)
make_sound(dog)
```

ការពន្យល់៖ `sound()` ត្រូវបានកំណត់ខុសគ្នានៅក្នុង Cat និង Dog, `make_sound()` អាចហៅ `<code>sound()</code>` លើវត្ថុណាមួយដែលមានមុខងារនេះ។

លទ្ធផល៖

```
ម៉ៅ!
វ៉ូហ្វ!
```

9.5 Encapsulation - ការបេចខ្ចប់

Encapsulation លាក់ព័ត៌មានខាងក្នុងរបស់វត្ថុ និងអនុញ្ញាតឱ្យចូលប្រើតាមវិធីសាស្ត្រជាក់លាក់ប៉ុណ្ណោះ។

ឧទាហរណ៍៖

```
class Student:
    def __init__(self, name):
        self.__grade = 85 # private attribute
    def get_grade(self):
        return self.__grade
    def set_grade(self, grade):
        if 0 <= grade <= 100:
            self.__grade = grade

student = Student("សុភី")
print(student.get_grade())
student.set_grade(90)
print(student.get_grade())
```

ការពន្យល់៖ __grade ជាលក្ខណៈឯកជន (private) ដែលមិនអាចចូលប្រើដោយផ្ទាល់, get_grade() ទាញតម្លៃ, set_grade() កែតម្លៃដោយមានលក្ខខណ្ឌ។

លទ្ធផល៖

```
85
90
```

9.6 Magic Methods (__str__, __len__, etc.) - វិធីសាស្ត្រអេម៉ង់

Magic methods ជាមុខងារពិសេសដែលចាប់ផ្តើម និងបញ្ចប់ដោយ __ ដើម្បីកំណត់ឥរិយាបថផ្ទាល់ខ្លួនរបស់វត្ថុ។

ឧទាហរណ៍៖

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
```

```

        return f"សៀវភៅ: {self.title}"

    def __len__(self):
        return self.pages

book = Book("Python សម្រាប់អ្នកចាប់ផ្តើម", 300)
print(book)
print(len(book))

```

ការពន្យល់៖ `__str__` កំណត់អ្វីដែលត្រូវបង្ហាញនៅពេលបោះពុម្ពវត្ថុ, `__len__` កំណត់ប្រវែងនៃវត្ថុនៅពេលប្រើ `len()`។

លទ្ធផល៖

```

សៀវភៅ: Python សម្រាប់អ្នកចាប់ផ្តើម
300

```

10. Exception Handling - ការគ្រប់គ្រងករណីលើកលែង

Exception Handling នៅក្នុង Python ជាវិធីសាស្ត្រដើម្បីដោះស្រាយកំហុស (errors) ដែលកើតឡើងអំឡុងពេលដំណើរការកម្មវិធី ដើម្បីការពារកម្មវិធីពីការគាំង។ ផ្នែកនេះនឹងបង្ហាញពីវិធីគ្រប់គ្រងករណីលើកលែង។

10.1 try-except Blocks - ប្លុក try-except

try-except ប្លុកប្រើដើម្បីសាកល្បងកូដដែលអាចមានកំហុស និងចាប់យកកំហុសប្រសិនបើវាកើតឡើង។

ឧទាហរណ៍៖

```

try:
    number = int(input("បញ្ចូលលេខ: "))
    result = 10 / number
    print(f"លទ្ធផល: {result}")
except ZeroDivisionError:
    print("មិនអាចបែងចែកដោយ 0 បានទេ!")

```

```
except ValueError:
    print("សូមបញ្ចូលលេខគត់!")
```

ការពន្យល់៖ try សាកល្បងកូដដែលអាចមានកំហុស, except ZeroDivisionError ចាប់កំហុសនៃការបែងចែកដោយ 0, except ValueError ចាប់កំហុសនៃការបញ្ចូលតម្លៃមិនមែនលេខ។

លទ្ធផល (បញ្ចូល 0)៖

មិនអាចបែងចែកដោយ 0 បានទេ!

លទ្ធផល (បញ្ចូល 'abc')៖

សូមបញ្ចូលលេខគត់!

10.2 Raising Exceptions (raise) - ការបង្កើតករណីលើកលែង

raise ប្រើដើម្បីបង្កើតកំហុសដោយចេតនា នៅពេលលក្ខខណ្ឌមួយមិនត្រូវបានបំពេញ។

ឧទាហរណ៍៖

```
def check_age(age):
    if age < 0:
        raise ValueError("អាយុមិនអាចតូចជាង 0 បានទេ!")
    print(f"អាយុរបស់អ្នកគឺ {age} ឆ្នាំ")

try:
    check_age(-5)
except ValueError as error:
    print(error)
```

ការពន្យល់៖ raise ValueError បង្កើតកំហុសប្រសិនបើអាយុតូចជាង 0, except ValueError as error ចាប់កំហុសនិងបង្ហាញសារ។

លទ្ធផល៖

អាយុមិនអាចតូចជាង 0 បានទេ!

10.3 Custom Exceptions - ករណីលើកលែងផ្ទាល់ខ្លួន

អ្នកអាចបង្កើត custom exceptions ដោយបង្កើតថ្នាក់ថ្មីដែលទទួលមរតកពី Exception។

ឧទាហរណ៍៖


```

class NegativeNumberError(Exception):
    pass

def divide(a, b):
    if b < 0:
        raise NegativeNumberError("មិនអាចបែងចែកដោយលេខអវិជ្ជមានបានទេ!")
    return a / b

try:
    result = divide(10, -2)
except NegativeNumberError as error:
    print(error)
except ZeroDivisionError:
    print("មិនអាចបែងចែកដោយ 0 បានទេ!")

```

ការពន្យល់៖ class NegativeNumberError(Exception) បង្កើតករណីលើកលែងផ្ទាល់ខ្លួន, raise NegativeNumberError បង្កើតកំហុសផ្ទាល់ខ្លួនប្រសិនបើ b អវិជ្ជមាន, except NegativeNumberError ចាប់កំហុសផ្ទាល់ខ្លួន។

លទ្ធផល៖

មិនអាចបែងចែកដោយលេខអវិជ្ជមានបានទេ!

11. Python Modules & Standard Libraries - ម៉ូឌុល និង បណ្ណាល័យស្តង់ដាររបស់ Python

Python Modules និង Standard Libraries ជាសំណុំនៃមុខងារដែលមានស្រាប់នៅក្នុង Python ដើម្បីជួយសម្រួលការងារផ្សេងៗដូចជា គណនា គ្រប់គ្រងកាលបរិច្ឆេទ ឬធ្វើការជាមួយប្រព័ន្ធប្រតិបត្តិការ។ ផ្នែកនេះនឹងបង្ហាញពីម៉ូឌុលសំខាន់ៗមួយចំនួន។

11.1 math (Mathematical Functions) - ម៉ូឌុល math (មុខងារគណិតវិទ្យា)

math module ផ្តល់មុខងារគណិតវិទ្យាដូចជា ឫសការ៉េ ស៊ីនុស និងការប្រមូលផ្តុំ។

ឧទាហរណ៍៖

```
import math

print(math.sqrt(16)) # ឬសការ៉េ
print(math.pi)      # តម្លៃ π
print(math.sin(math.pi / 2)) # ស៊ីនុស 90 ដឺក្រេ
```

ការពន្យល់៖ import math នាំចូលម៉ូឌុល math, math.sqrt() គណនាឬសការ៉េ, math.pi តម្លៃថេរ π, math.sin() គណនាស៊ីនុស (តម្លៃជា radian)។

លទ្ធផល៖

```
4.0
3.141592653589793
1.0
```

11.2 datetime (Date & Time) - ម៉ូឌុល datetime (កាលបរិច្ឆេទ និង ពេលវេលា)

datetime module ជួយគ្រប់គ្រងកាលបរិច្ឆេទ និងពេលវេលា។

ឧទាហរណ៍៖

```
from datetime import datetime

# ពេលវេលាបច្ចុប្បន្ន
now = datetime.now()
print(f"ថ្ងៃនេះ: {now.strftime('%Y-%m-%d %H:%M:%S')}")

# កាលបរិច្ឆេទជាក់លាក់
birthday = datetime(2000, 5, 15)
print(f"ថ្ងៃកំណើត: {birthday.strftime('%Y-%m-%d')}")
```

ការពន្យល់៖ from datetime import datetime នាំចូល datetime class, datetime.now() ទាញពេលវេលាបច្ចុប្បន្ន, strftime() ធ្វើទ្រង់ទ្រាយកាលបរិច្ឆេទទៅជាអក្សរ, datetime(2000, 5, 15) បង្កើតកាលបរិច្ឆេទជាក់លាក់។

លទ្ធផល (ឧទាហរណ៍)៖

ថ្ងៃនេះ: 2025-04-10 12:00:00

ថ្ងៃកំណើត: 2000-05-15

11.3 random (Generating Random Data) - ម៉ូឌុល random (បង្កើតទិន្នន័យចៃដន្យ)

random module បង្កើតលេខ ឬទិន្នន័យចៃដន្យសម្រាប់ការប្រើប្រាស់ផ្សេងៗ។

ឧទាហរណ៍៖

```
import random

print(random.randint(1, 10)) # លេខចៃដន្យ 1-10
print(random.choice(["ផ្លែប៉ោម", "ផ្លែចេក", "ផ្លែឪឡឹក"])) # ជ្រើសរើសចៃដន្យ
numbers = [1, 2, 3, 4, 5]
random.shuffle(numbers) # សាប៊ែរចៃដន្យ
print(numbers)
```

ការពន្យល់៖ random.randint(1, 10) បង្កើតលេខគត់ចៃដន្យរវាង 1 និង 10, random.choice() ជ្រើសរើសធាតុមួយចៃដន្យពីបញ្ជី, random.shuffle() សាប៊ែរលំដាប់ធាតុក្នុងបញ្ជី។

លទ្ធផល (ឧទាហរណ៍)៖

```
7
ផ្លែចេក
[3, 1, 5, 2, 4]
```

11.4 os (Interacting with OS) - ម៉ូឌុល os (ធ្វើអន្តរកម្មជាមួយប្រព័ន្ធប្រតិបត្តិការ)

os module អនុញ្ញាតឱ្យធ្វើអន្តរកម្មជាមួយប្រព័ន្ធប្រតិបត្តិការ ដូចជាគ្រប់គ្រងឯកសារ ឬថតឯកសារ។

ឧទាហរណ៍៖

```
import os

# ទាញថតបច្ចុប្បន្ន
print(os.getcwd())

# បង្កើតថតថ្មី
```

```
os.mkdir("test_folder")

# ប្តូរឈ្មោះថត
os.rename("test_folder", "new_folder")

# លុបថត
os.rmdir("new_folder")
```

ការពន្យល់៖ `os.getcwd()` ទាញផ្លូវថតបច្ចុប្បន្ន, `os.mkdir()` បង្កើតថតថ្មី, `os.rename()` ប្តូរឈ្មោះថត, `os.rmdir()` លុបថតទទេ។

លទ្ធផល (ឧទាហរណ៍)៖

```
/home/user/projects
```

11.5 sys (System-specific Parameters) - ម៉ូឌុល sys (ប៉ារ៉ាម៉ែត្រពិសេសស្របប្រព័ន្ធ)

`sys` module ផ្តល់ព័ត៌មាន និងគ្រប់គ្រងប៉ារ៉ាម៉ែត្រពិសេសស្របប្រព័ន្ធ Python។

ឧទាហរណ៍៖

```
import sys

# បង្ហាញជំនាន់ Python
print(sys.version)

# ទាញអាគុយម៉ង់ពី command line
print(sys.argv)

# ចាកចេញពីកម្មវិធី
# sys.exit(0) # បញ្ចប់ដោយជោគជ័យ (បានបញ្ចូលជា comment ដើម្បីបង្ហាញលទ្ធផល)
```

ការពន្យល់៖ `sys.version` បង្ហាញជំនាន់ Python, `sys.argv` ទាញបញ្ជីអាគុយម៉ង់ពី command line, `sys.exit()` បញ្ចប់កម្មវិធី (0 = ជោគជ័យ, ផ្សេងទៀត = បរាជ័យ)។

លទ្ធផល (ឧទាហរណ៍)៖

```
3.11.2 (main, Feb 7 2023, 23:15:45) [GCC 10.2.1]
['script.py', 'arg1', 'arg2']
```

12. Functional Programming - ការសរសេរកម្មវិធីបែបមុខងារ

Functional Programming ជាវិធីសាស្ត្រសរសេរកម្មវិធីដែលផ្ដោតលើការប្រើមុខងារ (functions) និងជៀសវាងការផ្លាស់ប្តូរស្ថានភាព (state)។ Python គាំទ្រឧបករណ៍សម្រាប់ការសរសេរបែបនេះដូចជា map, filter, និង reduce។

12.1 Map, Filter, Reduce - ម៉ែប ហ្វីលធី និង រីឌុស

map, filter, និង reduce ជាមុខងារដែលធ្វើការជាមួយបញ្ជីដោយបែបមុខងារ។

ឧទាហរណ៍៖

```
from functools import reduce

# map: អនុវត្តមុខងារលើធាតុនីមួយៗ
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
print(squared)

# filter: ជ្រើសរើសធាតុតាមលក្ខខណ្ឌ
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)

# reduce: បញ្ចូលធាតុទាំងអស់ទៅជាតម្លៃតែមួយ
sum_result = reduce(lambda x, y: x + y, numbers)
print(sum_result)
```

ការពន្យល់៖

- map(lambda x: x**2, numbers) ប្តូរធាតុនីមួយៗទៅជាការ៉េ,
- filter(lambda x: x % 2 == 0, numbers) ជ្រើសតែលេខគូ,
- reduce(lambda x, y: x + y, numbers) បូកធាតុទាំងអស់,

- from functools import reduce នាំចូល `<code>reduce</code>`

ពី `<code>functools</code>`។

លទ្ធផល៖

```
[1, 4, 9, 16]
[2, 4]
10
```

12.2 List Comprehensions - ការបង្កើតបញ្ជីបែបសង្ខេប

List comprehensions ជាវិធីសង្ខេបក្នុងការបង្កើតបញ្ជីដោយប្រើនិយាយកថាខ្លី និងមានប្រសិទ្ធភាព។

ឧទាហរណ៍៖

```
# បញ្ជីការ៉េ
squares = [x**2 for x in range(5)]
print(squares)

# បញ្ជីលេខគូ
evens = [x for x in range(10) if x % 2 == 0]
print(evens)
```

ការពន្យល់៖ `[x**2 for x in range(5)]` បង្កើតបញ្ជីការ៉េនៃលេខ 0-4, `[x for x in range(10) if x % 2 == 0]` បង្កើតបញ្ជីលេខគូពី 0-9។

លទ្ធផល៖

```
[0, 1, 4, 9, 16]
[0, 2, 4, 6, 8]
```

12.3 Generators & Iterators - ឧបករណ៍បង្កើត និង ឧបករណ៍រាប់

Generators និង Iterators ជួយគ្រប់គ្រងទិន្នន័យជាលំដាប់ដោយសន្សំសំចៃអង្គចងចាំ។

ឧទាហរណ៍៖

```
# Generator
def count_up_to(n):
    for i in range(n):
        yield i
```

```

counter = count_up_to(5)
print(list(counter))

# Iterator
numbers = iter([1, 2, 3, 4])
print(next(numbers))
print(next(numbers))

```

ការពន្យល់៖

- yield បង្កើត generator ដែលផ្តល់តម្លៃម្តងមួយៗ,
- count_up_to(5) ផ្តល់លេខ 0-4 តាមលំដាប់,
- iter() បង្កើត iterator ពីបញ្ជី,
- next() ទាញធាតុបន្ទាប់ពី iterator។

លទ្ធផល៖

```

[0, 1, 2, 3, 4]
1
2

```

13. Multithreading & Multiprocessing - ការដំណើរការច្រើនខ្សែ និង ច្រើនដំណើរការ

Multithreading និង Multiprocessing ជាវិធីសាស្ត្រដើម្បីដំណើរការកិច្ចការច្រើនក្នុងពេលតែមួយនៅក្នុង Python។ **threading** ប្រើសម្រាប់ខ្សែបន្តបន្ទាប់ (threads) ក្នុងដំណើរការតែមួយ ចំណែក **multiprocessing** ប្រើដំណើរការច្រើន (processes) ដើម្បីទាញយកអត្ថប្រយោជន៍ពី CPU ច្រើនស្នូល។

13.1 Threading in Python (threading module) - ការដំណើរការច្រើនខ្សែ (ម៉ូឌុល threading)

threading module អនុញ្ញាតឱ្យដំណើរការកិច្ចការច្រើនក្នុងខ្សែបន្តបន្ទាប់ (threads) ដែលស័ក្តិសមសម្រាប់ការងារ I/O-bound (ឧ. ការអាន/សរសេរឯកសារ)។

ឧទាហរណ៍៖

```

import threading
import time

def task(name):
    print(f"ខ្សែ {name} ចាប់ផ្តើម")
    time.sleep(2) # ក្លែងធ្វើការងារដែលចំណាយពេល
    print(f"ខ្សែ {name} បញ្ចប់")

# បង្កើតខ្សែ
t1 = threading.Thread(target=task, args=("ទី១",))
t2 = threading.Thread(target=task, args=("ទី២",))

# ចាប់ផ្តើមខ្សែ
t1.start()
t2.start()

# រង់ចាំខ្សែបញ្ចប់
t1.join()
t2.join()

print("ការងារទាំងអស់បញ្ចប់")

```

ការពន្យល់៖

- import threading នាំចូលម៉ូឌុល threading,
- threading.Thread(target=task, args=...) បង្កើតខ្សែថ្មីជាមួយមុខងារ target,
- start() ចាប់ផ្តើមខ្សែ,
- join() រង់ចាំខ្សែបញ្ចប់មុននឹងបន្ត។

លទ្ធផល៖

```

ខ្សែ ទី១ ចាប់ផ្តើម
ខ្សែ ទី២ ចាប់ផ្តើម
(រង់ចាំ 2 វិនាទី)
ខ្សែ ទី១ បញ្ចប់
ខ្សែ ទី២ បញ្ចប់
ការងារទាំងអស់បញ្ចប់

```

13.2 Parallel Processing (multiprocessing module) - ការដំណើរការស្របគ្នា (ម៉ូឌុល multiprocessing)

multiprocessing module ប្រើដំណើរការច្រើន (processes) ដើម្បីដំណើរការកិច្ចការស្របគ្នា ស័ក្តិសមសម្រាប់ការងារ CPU-bound (ឧ. ការគណនាធំៗ)។

ឧទាហរណ៍៖

```
import multiprocessing
import time

def square_number(n):
    print(f"ដំណើរការ {multiprocessing.current_process().name} គណនា {n}")
    time.sleep(1) # ក្លែងធ្វើការងារដែលចំណាយពេល
    return n**2

if __name__ == "__main__":
    numbers = [1, 2, 3, 4]

    # បង្កើត Pool នៃដំណើរការ
    with multiprocessing.Pool(processes=2) as pool:
        results = pool.map(square_number, numbers)

    print(f"លទ្ធផល: {results}")
```

ការពន្យល់៖

- import multiprocessing នាំចូលម៉ូឌុល multiprocessing,
- multiprocessing.Pool(processes=2) បង្កើតក្រុមដំណើរការចំនួន 2,
- pool.map() អនុវត្តមុខងារ square_number លើធាតុនីមួយៗស្របគ្នា,
- if __name__ == '__main__': ចាំបាច់សម្រាប់ multiprocessing នៅលើ Windows។

លទ្ធផល៖

```
ដំណើរការ PoolWorker-1 គណនា 1**2
ដំណើរការ PoolWorker-2 គណនា 2**2
(រង់ចាំ 1 វិនាទី)
ដំណើរការ PoolWorker-1 គណនា 3**2
ដំណើរការ PoolWorker-2 គណនា 4**2
(រង់ចាំ 1 វិនាទី)
លទ្ធផល: [1, 4, 9, 16]
```

14. Database Handling - ការគ្រប់គ្រងមូលដ្ឋានទិន្នន័យ

Database Handling នៅក្នុង Python អនុញ្ញាតឱ្យធ្វើអន្តរកម្មជាមួយមូលដ្ឋានទិន្នន័យដូចជា SQLite, MySQL, និង PostgreSQL ដើម្បីរក្សាទុក និងទាញយកទិន្នន័យ។ ផ្នែកនេះនឹងបង្ហាញពីរបៀបប្រើម៉ូឌុលសម្រាប់មូលដ្ឋានទិន្នន័យទាំងនេះ។

14.1 SQLite (sqlite3 module) - SQLite (ម៉ូឌុល sqlite3)

sqlite3 module ជាម៉ូឌុលស្តង់ដារសម្រាប់ធ្វើការជាមួយ SQLite ដែលជាមូលដ្ឋានទិន្នន័យទម្ងន់ស្រាល និងមិនតម្រូវឱ្យមាន server ដាច់ដោយឡែក។

ឧទាហរណ៍៖

```
import sqlite3

# ភ្ជាប់ទៅមូលដ្ឋានទិន្នន័យ
conn = sqlite3.connect("students.db")
cursor = conn.cursor()

# បង្កើតតារាង
cursor.execute("CREATE TABLE IF NOT EXISTS students (id INTEGER PRIMARY KEY, name TEXT, age INTEGER)")

# បញ្ចូលទិន្នន័យ
cursor.execute("INSERT INTO students (name, age) VALUES (?, ?)", ("សុខា", 25))

# ទាញទិន្នន័យ
cursor.execute("SELECT * FROM students")
print(cursor.fetchall())

# រក្សាទុក និងបិទ
conn.commit()
conn.close()
```

ការពន្យល់៖

- `sqlite3.connect()` ភ្ជាប់ទៅឯកសារ `.db` (បង្កើតបើមិនទាន់មាន),
- `cursor.execute()` ប្រតិបត្តិពាក្យបញ្ជា SQL,

- INSERT INTO បញ្ចូលទិន្នន័យ,
- fetchall() ទាញទិន្នន័យទាំងអស់,
- conn.commit() រក្សាទុកការផ្លាស់ប្តូរ,
- conn.close() បិទការតភ្ជាប់។

លទ្ធផល (ឧទាហរណ៍)៖

```
[(1, 'សុខា', 25)]
```

14.2 MySQL (mysql-connector) - MySQL (mysql-connector)

mysql-connector-python ជាម៉ូឌុលសម្រាប់ភ្ជាប់ Python ទៅ MySQL server។ ត្រូវតំឡើងដោយប្រើ
 pip install mysql-connector-python។

ឧទាហរណ៍៖

```
import mysql.connector

# ភ្ជាប់ទៅ MySQL server
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password",
    database="school"
)
cursor = conn.cursor()

# បង្កើតតារាង
cursor.execute("CREATE TABLE IF NOT EXISTS students (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50), age INT)")

# បញ្ចូលទិន្នន័យ
cursor.execute("INSERT INTO students (name, age) VALUES (%s, %s)", ("សុខា", 25))

# ទាញទិន្នន័យ
cursor.execute("SELECT * FROM students")
print(cursor.fetchall())

# រក្សាទុក និងបិទ
```

```
conn.commit()
conn.close()
```

ការពន្យល់៖

- `mysql.connector.connect()` ភ្ជាប់ទៅ MySQL server ដោយបញ្ជាក់ `host`, `user`, `password`, និង `database`,
- `cursor.execute()` ប្រតិបត្តិ SQL,
- `%s` ជាកន្លែងបញ្ចូលតម្លៃដោយសុវត្ថិភាព,
- `fetchall()` ទាញលទ្ធផលទាំងអស់។

លទ្ធផល (ឧទាហរណ៍)៖

```
[(1, 'សុភី', 30)]
```

14.3 PostgreSQL (psycopg2) - PostgreSQL (psycopg2)

`psycopg2` ជាម៉ូឌុលសម្រាប់ភ្ជាប់ Python ទៅ PostgreSQL server។ ត្រូវតំឡើងវាដោយប្រើ `pip install psycopg2`។

ឧទាហរណ៍៖

```
import psycopg2

# ភ្ជាប់ទៅ PostgreSQL server
conn = psycopg2.connect(
    host="localhost",
    database="school",
    user="postgres",
    password="your_password"
)
cursor = conn.cursor()

# បង្កើតតារាង
cursor.execute("CREATE TABLE IF NOT EXISTS students (id SERIAL PRIMARY KEY, name VARCHAR(100), age INT)")

# បញ្ចូលទិន្នន័យ
cursor.execute("INSERT INTO students (name, age) VALUES (%s, %s)", ('លីណា', 25))

# ទាញទិន្នន័យ
cursor.execute("SELECT * FROM students")
```

```
print(cursor.fetchall())
```

```
# រក្សាទុក និងបិទ  
conn.commit()  
conn.close()
```

ការពន្យល់៖

- `psycopg2.connect()` ភ្ជាប់ទៅ PostgreSQL server ដោយបញ្ជាក់ host, database, user, password,
- `SERIAL` ជាប្រភេទទិន្នន័យសម្រាប់ id ដែលបង្កើនដោយស្វ័យប្រវត្តិ,
- `cursor.execute()` ប្រតិបត្តិ SQL,
- `fetchall()` ទាញទិន្នន័យទាំងអស់។

លទ្ធផល (ឧទាហរណ៍)៖

```
[(1, 'សីណា', 22)]
```

15. Web Development with Python - ការអភិវឌ្ឍគេហទំព័រជាមួយ Python

ការអភិវឌ្ឍគេហទំព័រជាមួយ Python អនុញ្ញាតឱ្យអ្នកបង្កើតគេហទំព័រនិងកម្មវិធីគេហទំព័រដោយប្រើភាសា Python។ ផ្នែកនេះនឹងផ្ដោតលើក្របខ័ណ្ឌពីរដ៏ពេញនិយមគឺ Flask និង Django ដែលជួយសម្រួលដល់ការអភិវឌ្ឍគេហទំព័រ។

15.1 Introduction to Flask & Django - ការណែនាំអំពី Flask និង Django

Flask ជាក្របខ័ណ្ឌទម្ងន់ស្រាលសម្រាប់បង្កើតកម្មវិធីគេហទំព័រតូចៗ ឬ API ដែលផ្តល់ភាពបត់បែនខ្ពស់។ Django ជាក្របខ័ណ្ឌពេញលេញជាង ដែលស័ក្តិសមសម្រាប់គម្រោងធំៗ ដោយមានមុខងារជាច្រើនដូចជា ORM, admin panel និងសុវត្ថិភាព។ ទាំងពីរត្រូវតែដំឡើងដោយប្រើ `pip install flask` និង `pip install django`។

15.2 Creating a Simple API with Flask - ការបង្កើត API សាមញ្ញជាមួយ Flask

នេះគឺជាឧទាហរណ៍នៃការបង្កើត API សាមញ្ញដោយប្រើ Flask ដើម្បីឆ្លើយតបនឹងសំណើ HTTP។

ឧទាហរណ៍៖

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return jsonify({"message": "សូមស្វាគមន៍មកកាន់ Flask API!"})

@app.route('/api/user/<name>')
def get_user(name):
    return jsonify({"name": name, "age": 25})

if __name__ == '__main__':
    app.run(debug=True)
```

ការពន្យល់៖

- Flask(__name__) បង្កើតកម្មវិធី Flask,
- @app.route('/') កំណត់ផ្លូវសម្រាប់ទំព័រដើម,
- jsonify ប្តូរទិន្នន័យទៅជា JSON,
- /api/user/<name> ផ្លូវដែលទទួលបានពីអ្នកប្រើប្រាស់ឈ្មោះ,
- app.run(debug=True) ដំណើរការកម្មវិធីជាមួយរបៀប debug។

លទ្ធផល (ឧទាហរណ៍)៖ បើចូលទៅ <http://localhost:5000/api/user/សុខា>

```
{"name": "សុខា", "age": 25}
```

15.3 Building Web Applications with Django - ការបង្កើតកម្មវិធីគេហទំព័រជាមួយ Django

Django អនុញ្ញាតឱ្យអ្នកបង្កើតកម្មវិធីគេហទំព័រដែលមានរចនាសម្ព័ន្ធច្បាស់លាស់។ នេះគឺជាឧទាហរណ៍មួយសាមញ្ញ។

ឧទាហរណ៍៖ (បន្ទាប់ពីបង្កើតគម្រោងជាមួយ `django-admin startproject myproject` និង `app` ជាមួយ `python manage.py startapp myapp`)

```
# myapp/views.py
from django.http import JsonResponse

def home(request):
```

```

        return JsonResponse({"message": "សូមស្វាគមន៍មកកាន់ Django!"})

# myproject/urls.py
from django.urls import path
from myapp import views

urlpatterns = [
    path('', views.home, name='home'),
]

# ដំណើរការដោយ python manage.py runserver

```

ការពន្យល់៖

- views.py កំណត់អនុគមន៍សម្រាប់ឆ្លើយតបសំណើ,
- JsonResponse ធ្វើទិន្នន័យជា JSON,
- urls.py ភ្ជាប់ផ្លូវទៅនឹងអនុគមន៍,
- runserver ចាប់ផ្តើម server នៅ localhost:8000។

លទ្ធផល (ឧទាហរណ៍)៖ បើចូលទៅ <http://localhost:8000/>

```

{"message": "សូមស្វាគមន៍មកកាន់ Django!"}

```

16. Data Science & Machine Learning - វិទ្យាសាស្ត្រទិន្នន័យ និងការរៀនសូត្រដោយម៉ាស៊ីន

វិទ្យាសាស្ត្រទិន្នន័យ និងការរៀនសូត្រដោយម៉ាស៊ីនជាផ្នែកសំខាន់នៃការវិភាគទិន្នន័យ និងការបង្កើតកំរិតដែលអាចទស្សន៍ទាយបាន។ Python ផ្តល់នូវបណ្ណាល័យដ៏មានឥទ្ធិពលដូចជា NumPy, Pandas, Matplotlib, Seaborn និង Scikit-learn ដើម្បីគ្រប់គ្រងទិន្នន័យ និងបង្កើតក្បួនដោះស្រាយ។

16.1 NumPy (Numerical Computing) - NumPy (ការគណនាជាលេខ)

NumPy ជាបណ្ណាល័យសម្រាប់ការគណនាជាលេខ ដែលផ្តល់នូវការគាំទ្រសម្រាប់អារេច្រើនវិមាត្រ និងប្រតិបត្តិការគណិតវិទ្យា។ ត្រូវតែឡើងវិញដោយប្រើ `pip install numpy`។

ឧទាហរណ៍៖

```
import numpy as np

# បង្កើតអារេ
array = np.array([1, 2, 3, 4, 5])
print("អារេ:", array)

# ប្រតិបត្តិការគណិតវិទ្យា
squared = np.square(array)
print("លេខការ៉េ:", squared)
```

ការពន្យល់៖

- np.array() បង្កើតអារេ NumPy,
- np.square() គណនាការ៉េនៃធាតុនីមួយៗក្នុងអារេ។

លទ្ធផល (ឧទាហរណ៍)៖

```
អារេ: [1 2 3 4 5]
លេខការ៉េ: [ 1  4  9 16 25]
```

16.2 Pandas (Data Analysis) - Pandas (ការវិភាគទិន្នន័យ)

Pandas ជាបណ្តាញសម្រាប់ការវិភាគទិន្នន័យ ដែលផ្តល់នូវរចនាសម្ព័ន្ធទិន្នន័យដូចជា Series និង DataFrame។ ត្រូវតំឡើងវាដោយប្រើ `pip install pandas`។

ឧទាហរណ៍៖

```
import pandas as pd

# បង្កើត DataFrame
data = {'ឈ្មោះ': ['សុខា', 'សុភី'], 'អាយុ': [25, 30]}
df = pd.DataFrame(data)
print("DataFrame:")
print(df)

# ចម្រោះទិន្នន័យ
print("អាយុលើសពី 25:")
print(df[df['អាយុ'] > 25])
```

ការពន្យល់៖

- pd.DataFrame() បង្កើតតារាងទិន្នន័យ,

- df['អាយុ'] > 25 ចម្រោះជួរដែលមានអាយុលើសពី 25។

លទ្ធផល (ឧទាហរណ៍)៖

```
DataFrame:
  ឈ្មោះ អាយុ
0  សុខា      25
1  សុភី      30
អាយុលើសពី 25:
  ឈ្មោះ អាយុ
1  សុភី      30
```

16.3 Matplotlib & Seaborn (Data Visualization) - Matplotlib និង Seaborn (ការមើលឃើញទិន្នន័យ)

Matplotlib និង Seaborn ជាបណ្តាញសម្រាប់បង្កើតក្រាបទិន្នន័យ។ Matplotlib ផ្តល់មូលដ្ឋានគ្រឹះ ខណៈ Seaborn ធ្វើឱ្យក្រាបកាន់តែស្រស់ស្អាត។ ត្រូវតំឡើងដោយ `pip install matplotlib seaborn`។

ឧទាហរណ៍៖

```
import matplotlib.pyplot as plt
import seaborn as sns

# ទិន្នន័យ
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

# គូសក្រាបជាមួយ Matplotlib
plt.plot(x, y, marker='o')
plt.title("ក្រាបសាមញ្ញ")
plt.show()

# គូសក្រាបជាមួយ Seaborn
sns.barplot(x=x, y=y)
plt.title("ក្រាបបារសាមញ្ញ")
plt.show()
```

ការពន្យល់៖

- `plt.plot()` គូសក្រាបបន្ទាត់,
- `sns.barplot()` គូសក្រាបបារ,
- `plt.show()` បង្ហាញក្រាប។

16.4 Scikit-learn (Machine Learning) - Scikit-learn (ការរៀនសូត្រដោយម៉ាស៊ីន)

Scikit-learn ជាបណ្តាញសម្រាប់ការរៀនសូត្រដោយម៉ាស៊ីន ដែលផ្តល់ឧបករណ៍សម្រាប់ការចាត់ថ្នាក់ និងការទស្សន៍ទាយ។ ត្រូវតែឡើងដោយ `pip install scikit-learn`។

ឧទាហរណ៍៖

```
from sklearn.linear_model import LinearRegression
import numpy as np

# ទិន្នន័យ
X = np.array([[1], [2], [3], [4]])
y = np.array([2, 4, 5, 4])

# បង្កើត និងហ្វីតគំរូ
model = LinearRegression()
model.fit(X, y)

# ទស្សន៍ទាយ
prediction = model.predict(np.array([[5]]))
print("ការទស្សន៍ទាយ:", prediction)
```

ការពន្យល់៖

- `LinearRegression()` បង្កើតគំរូតម្រង់ជួរ,
- `model.fit()` ហ្វីតគំរូជាមួយទិន្នន័យ,
- `model.predict()` ទស្សន៍ទាយតម្លៃថ្មី។

លទ្ធផល (ឧទាហរណ៍)៖

```
ការទស្សន៍ទាយ: [4.8]
```

17. Web Scraping - ការប្រមូលទិន្នន័យពីគេហទំព័រ

Web Scraping គឺជាបច្ចេកទេសសម្រាប់ទាញយកទិន្នន័យពីគេហទំព័រដោយប្រើ Python។ ផ្នែកនេះនឹងបង្ហាញពីរបៀបប្រើបណ្តាញដូចជា `requests`, `BeautifulSoup` និង `Selenium` ដើម្បីប្រមូលទិន្នន័យពីគេហទំព័រវិបតិរន្ត និងថាមវន្ត។

17.1 Using requests & BeautifulSoup - ការប្រើ requests និង BeautifulSoup

requests និង BeautifulSoup ជាឧបករណ៍ដ៏ពេញនិយមសម្រាប់ប្រមូលទិន្នន័យពីគេហទំព័រវិប។ requests ប្រើសម្រាប់ទាញ HTML ហើយ BeautifulSoup ប្រើសម្រាប់វិភាគ និងស្វែងរកទិន្នន័យ។ ត្រូវតែឡើងដោយ `pip install requests beautifulsoup4`។

ឧទាហរណ៍៖

```
import requests
from bs4 import BeautifulSoup

# ទាញទំព័រ
url = "https://example.com"
response = requests.get(url)

# វិភាគ HTML
soup = BeautifulSoup(response.text, 'html.parser')

# ស្វែងរកទិន្នន័យ
title = soup.find('h1').text
print("ចំណងជើង:", title)
```

ការពន្យល់៖

- `requests.get()` ទាញ HTML ពី URL,
- `BeautifulSoup()` វិភាគ HTML,
- `soup.find('h1')` ស្វែងរកធាតុ `h1` ដំបូង និងទាញអត្ថបទ។

លទ្ធផល (ឧទាហរណ៍)៖

```
ចំណងជើង: Example Domain
```

17.2 Scraping Dynamic Websites with Selenium - ការប្រមូលទិន្នន័យពីគេហទំព័រថាមវន្តជាមួយ Selenium

Selenium ជាឧបករណ៍សម្រាប់ប្រមូលទិន្នន័យពីគេហទំព័រថាមវន្តដែលប្រើ JavaScript ដើម្បីបង្កើតមាតិកា។ វាអនុញ្ញាតឱ្យធ្វើអន្តរកម្មជាមួយទំព័រដូចជាមនុស្ស។ ត្រូវតែឡើងដោយ `pip install selenium` និងទាញយក WebDriver (ឧ. `ChromeDriver`)។

ឧទាហរណ៍៖

```

from selenium import webdriver
from selenium.webdriver.common.by import By

# ចាប់ផ្តើម WebDriver
driver = webdriver.Chrome() # ត្រូវបញ្ជាក់ផ្លូវទៅ ChromeDriver បើចាំបាច់
driver.get("https://example.com")

# ស្វែងរកទិន្នន័យ
title = driver.find_element(By.TAG_NAME, 'h1').text
print("ចំណងជើង:", title)

# បិទ WebDriver
driver.quit()

```

ការពន្យល់៖

- webdriver.Chrome() ចាប់ផ្តើមកម្មវិធីរុករក Chrome,
- driver.get() បើក URL,
- find_element(By.TAG_NAME, 'h1') ស្វែងរកធាតុ h1 និងទាញអត្ថបទ,
- driver.quit() បិទកម្មវិធីរុករក។

លទ្ធផល (ឧទាហរណ៍)៖

ចំណងជើង: Example Domain

18. Cybersecurity & Ethical Hacking - សន្តិសុខបច្ចេកវិទ្យា និងការវាយលុកប្រកបដោយក្រមសីលធម៌

Cybersecurity និង Ethical Hacking ជាផ្នែកសំខាន់ក្នុងការការពារប្រព័ន្ធ និងស្វែងរកចំណុចខ្សោយដោយប្រើបច្ចេកទេសស្របច្បាប់។ Python ផ្តល់ឧបករណ៍ដូចជា hashlib, cryptography និង socket module ដើម្បីអភិវឌ្ឍកម្មវិធីសន្តិសុខ និងសាកល្បងបណ្តាញ។

18.1 Hashing & Cryptography (hashlib, cryptography) - ការបម្លែងជាសញ្ញាសម្ងាត់ និងការអ៊ិនគ្រីប (hashlib, cryptography)

hashlib ជាម៉ូឌុលស្តង់ដារសម្រាប់បម្លែងទិន្នន័យជាសញ្ញាសម្ងាត់ (hashing) ដូចជា MD5 ឬ SHA-256។ cryptography ជាបណ្ណាល័យសម្រាប់ការអ៊ិនគ្រីប និងឌីគ្រីបទិន្នន័យ។ ត្រូវតែឡើង cryptography ដោយ pip install cryptography។

ឧទាហរណ៍៖

```
import hashlib
from cryptography.fernet import Fernet

# Hashing ជាមួយ hashlib
password = "mysecret"
hash_object = hashlib.sha256(password.encode())
print("SHA-256 Hash:", hash_object.hexdigest())

# Encryption ជាមួយ cryptography
key = Fernet.generate_key()
cipher = Fernet(key)
encrypted = cipher.encrypt(b"សូមស្វាគមន៍!")
print("ទិន្នន័យអ៊ិនគ្រីប:", encrypted)

# Decryption
decrypted = cipher.decrypt(encrypted)
print("ទិន្នន័យឌីគ្រីប:", decrypted.decode())
```

ការពន្យល់៖

- hashlib.sha256() បម្លែងទិន្នន័យជា hash SHA-256,
- Fernet.generate_key() បង្កើតកូនសោសម្រាប់អ៊ិនគ្រីប,
- cipher.encrypt() អ៊ិនគ្រីបទិន្នន័យ,
- cipher.decrypt() ឌីគ្រីបទិន្នន័យ។

លទ្ធផល (ឧទាហរណ៍)៖

```
SHA-256 Hash: 91fdeb5b4f6e7f90f3a3e9d5e6e8e9e4b5e8e5e6e8e9e4b5e8e5e6e8e9
ទិន្នន័យអ៊ិនគ្រីប: gAAAAAB... (ខុសគ្នារាល់ដង)
ទិន្នន័យឌីគ្រីប: សូមស្វាគមន៍!
```

18.2 Network Programming (socket module) - ការសរសេរកម្មវិធីបណ្តាញ (socket module)

socket module ជាម៉ូឌុលស្តង់ដារសម្រាប់ការសរសេរកម្មវិធីបណ្តាញ ដែលអនុញ្ញាតឱ្យភ្ជាប់ និងធ្វើទិន្នន័យរវាងកុំព្យូទ័រ។ វាអាចប្រើសម្រាប់សាកល្បងសន្តិសុខបណ្តាញ ឬបង្កើត server/client។

ឧទាហរណ៍៖ (Server និង Client សាមញ្ញ)

```
# Server
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('localhost', 12345))
server.listen(1)
print("Server កំពុងរង់ចាំ...")

conn, addr = server.accept()
print("ភ្ជាប់ពី:", addr)
data = conn.recv(1024)
print("ទទួល:", data.decode())
conn.close()

# Client (ដំណើរការដាច់ដោយឡែក)
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('localhost', 12345))
client.send("ស្អីពី Client!".encode())
client.close()
```

ការពន្យល់៖

- `socket.socket()` បង្កើត socket សម្រាប់ទំនាក់ទំនង,
- `server.bind()` ភ្ជាប់ server ទៅ port,
- `server.listen()` រង់ចាំការភ្ជាប់,
- `conn.recv()` ទទួលទិន្នន័យ,
- `client.connect()` ភ្ជាប់ទៅ server,
- `client.send()` ផ្ញើទិន្នន័យ។

លទ្ធផល (ឧទាហរណ៍)៖

```
Server កំពុងរង់ចាំ...
ភ្ជាប់ពី: ('127.0.0.1', 54321)
ទទួល: ស្អីពី Client!
```

អំពីយើង

CodeKhmerLearning ផ្តល់ឱ្យអ្នកនូវវគ្គសិក្សាពិសេសក្នុងការរៀនកូដភាសាខ្មែរ។ យើងមានក្រុមការងារព្រមមានជំនាញក្នុងការអភិវឌ្ឍកម្មវិធី និងវេបសាយ។

© 2025 CodeKhmerLearning. All Rights Reserved.

តំណភ្ជាប់រហ័ស

ផ្ទាំងដើម
វគ្គសិក្សា
អំពីយើង
ទំនាក់ទំនង

ទំនាក់ទំនង

សូមទំនាក់ទំនងមកកាន់យើងក្តី:

Email: thymuoyhak.biu@gmail.com

Phone: +855 888 052 812



© 2025 CodeKhmerLearning. All rights reserved.